

Continuous state spaces

Example of continuous state space applications

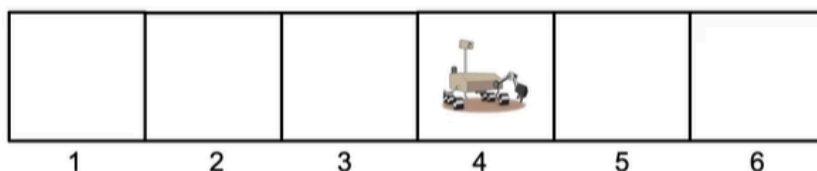
Many robotic control applications, including the lunar lander application that you work on in the practice lab, have continuous state spaces.

Let's take a look at what that means and how to generalize the concept we've talked about to these continuous state spaces.

A simplify Mars rover example we use, I use a discrete set of states, and what that means is that simplify Mars rover could only be in one of six possible positions. But most robots can be in more than one of six or any discrete number of positions, instead, they can be in any of a very large number of continuous value positions. For example, if the Mars rover could be anywhere on a line, so its position was indicated by a number ranging from 0-6 kilometers where any number in between is valid.

Discrete vs Continuous State

Discrete State:



Continuous State:



between zero and 360 degrees.

$$s = \begin{bmatrix} x \\ y \\ \theta \\ \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix}$$

That would be an example of a continuous state space, because the position would be represented by a number such as that is 2.7 kilometers along or 4.8 kilometers or any other number between zero and six. Let's look at another

example. I'm going to use for this example, the application of controlling a car or a truck.

Here's a toy car, or actually a toy truck. This one belongs to my daughter. If you're building a self-driving car or self-driving truck and you want to control this to drive smoothly, then the state of this truck might include a few numbers such as this, x position, its y position, maybe it's orientation. What way is it facing?

Assuming the truck stays on the ground, you probably don't need to worry about how tall is, how high up it is. This state will include x, y, and its angle θ , as well as maybe its speeds in x-direction, the speed in the y-direction, and how quickly it's turning. Is it turning at one degree per second or is it turning at 30 degrees per second or is it turning really quickly at 90 degrees per second?

For a truck or a car, the state might include not just one number, like how many kilometers of this along on this line, but they might include six numbers, x position, y position, orientation, which I'm going to denote using Greek alphabet θ , as well as its velocity in the x-direction, which I will denote using $\dot{x}$, so that means how quickly is this x-coordinate changing, $\dot{y}$ how quickly is the y coordinate changing, and then finally, $\dot{\theta}$, which is how quickly is the angle of the car changing. Whereas for the 60 Mars rover example, the state was just one of six possible numbers. It could be one, two, three, four, five or six.

For the car, the state would comprise this vector of six numbers, and any of these numbers can take on any value within its valid range. For example, θ should range between zero and 360 degrees.

Explanation of Bellman Equation

$$Q(s, a) = R(s) + \gamma \max_{a'} Q(s', a')$$

100	100	50	12.5	25	6.25	12.5	10	6.25	20	40	40
100	0	0	0	0	0	0	0	0	0	40	40
1	2	3	4	5	6						

$$\begin{aligned}
 Q(4, \leftarrow) &= 0 + (0.5)0 + (0.5)^2 0 + (0.5)^3 100 \\
 &= R(4) + (0.5) [0 + (0.5) 0 + (0.5)^2 100] \\
 &= R(4) + (0.5) \max_{a'} Q(3, a')
 \end{aligned}$$

To summarize, the state of the helicopter includes its position in the x, y, z direction, is positioned in the east-west direction, y is height above ground, and also the roll, the pitch, and also that yaw of helicopter and its speed in x, y, z,

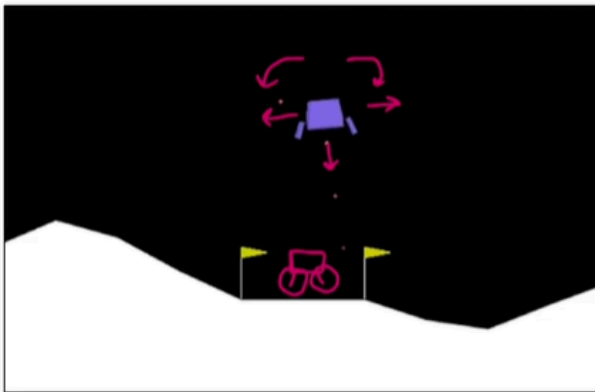
Autonomous Helicopter



$$s = \begin{bmatrix} x \\ y \\ z \\ \phi \\ \theta \\ \omega \\ \dot{x} \\ \dot{y} \\ \dot{z} \\ \dot{\phi} \\ \dot{\theta} \\ \dot{\omega} \end{bmatrix}$$

Lunar lander

Lunar Lander



actions:

do nothing

left thruster

main thruster

right thruster

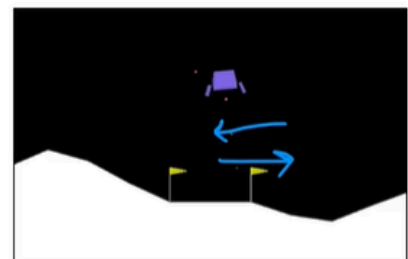
$$s = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \theta \\ \dot{\theta} \\ l \\ r \end{bmatrix}$$

0 or 1

We're going to have two other variables in the state vector which we call l and r . Which corresponds to whether the left leg is grounded, meaning whether or not the left leg is sitting on the ground as well as r which corresponds to whether or not the right leg is sitting on the ground.

Reward Function

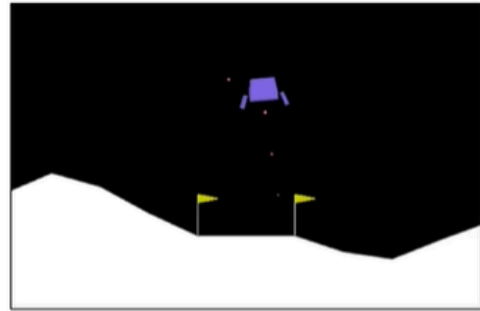
- Getting to landing pad: 100 – 140
- Additional reward for moving toward/away from pad.
- Crash: -100
- Soft landing: +100
- Leg grounded: +10
- Fire main engine: -0.3
- Fire side thruster: -0.03



Lunar Lander Problem

Learn a policy π that, given

$$s = \begin{bmatrix} x \\ y \\ \dot{x} \\ \dot{y} \\ \theta \\ \dot{\theta} \\ l \\ r \end{bmatrix}$$



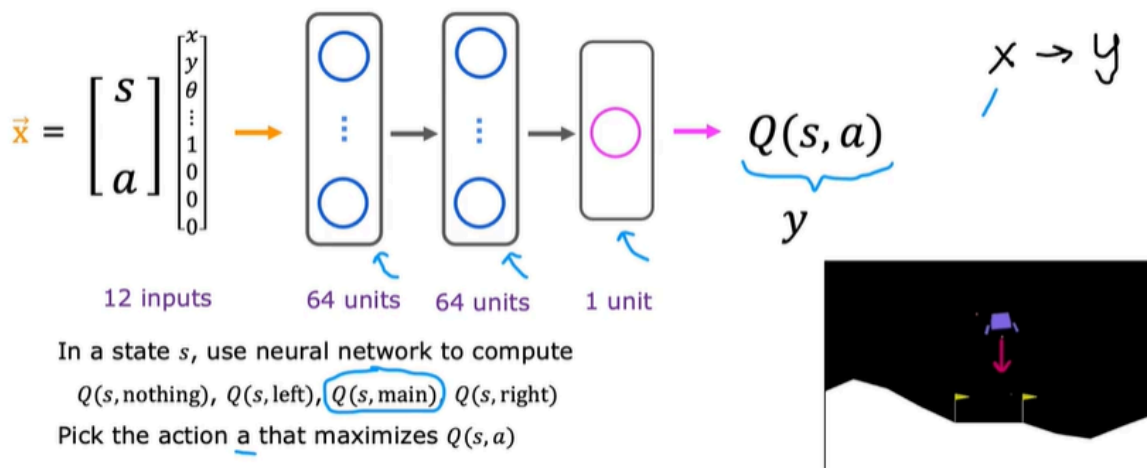
picks action $a = \pi(s)$ so as to maximize the return.

$$\gamma = 0.985$$

Learning the state-value function

Let's see how we can use reinforcement learning to control the Lunar Lander or for other reinforcement learning problems. The key idea is that we're going to train a neural network to compute or to approximate the state action value function $Q(s, a)$ and that in turn will let us pick good actions.

Deep Reinforcement Learning



$Q(s, \text{main}) \rightarrow$ fire the main engine

The question becomes, how do you train a neural network to output $Q(s, a)$? It turns out the approach will be to use Bellman's equations to create a training set with lots of examples, x and y , and then we'll use supervised learning

exactly as you learned in the second course when we talked about neural networks.

To learn using supervised learning, a mapping from x to y , that is a mapping from the state action pair to this target value $Q(s, a)$. But how do you get the training set with values for x and y that you can then train a neural network on?

Create training example

But it turns out that when you don't know what is the Q function, you can start off with taking a totally random guess. What is the Q function? We'll see on the next slide that the algorithm will work nonetheless. But in every step Q here is just going to be some guess. They'll get better over time it turns out of what is the actual Q function?

Bellman Equation

$$Q(\underbrace{s, a}_x) = R(s) + \underbrace{\gamma \max_{a'} Q(s', a')}_y$$

$f_{w, \beta}(x) \approx y$
 $(s, a, R(s), s')$

$(s^{(1)}, a^{(1)}, R(s^{(1)}), s'^{(1)}) \leftarrow$
 $(s^{(2)}, a^{(2)}, R(s^{(2)}), s'^{(2)}) \leftarrow$
 $(s^{(3)}, a^{(3)}, R(s^{(3)}), s'^{(3)}) \leftarrow$

$y^{(1)} = R(s^{(1)}) + \gamma \max_{a'} Q(s'^{(1)}, a')$
 $y^{(2)} = R(s^{(2)}) + \gamma \max_{a'} Q(s'^{(2)}, a')$

x	y
$x^{(1)} = (s^{(1)}, a^{(1)})$	$y^{(1)}$
$x^{(2)} = (s^{(2)}, a^{(2)})$	$y^{(2)}$
$x^{(10,000)}$	$y^{(10,000)}$

What we'll see later is that we'll actually take this training set where the x 's are inputs with 12 features and the y 's are just numbers. We'll train a neural network with, the mean squared error loss to try to predict y as a function of the input x .

Learning Algorithm

Initialize neural network randomly as guess of $Q(s, a)$.

Repeat {

Take actions in the lunar lander. Get $(s, a, R(s), s')$.

Store 10,000 most recent $(s, a, R(s), s')$ tuples.

Replay Buffer

Train neural network:

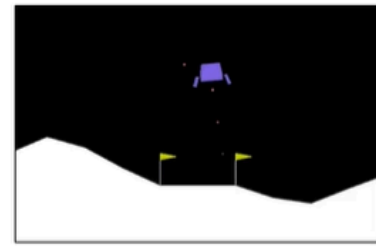
Create training set of 10,000 examples using

$$x = (s, a) \text{ and } y = R(s) + \gamma \max_{a'} Q(s', a')$$

Train Q_{new} such that $Q_{new}(s, a) \approx y$.

Set $Q = Q_{new}$.

$$f_{w,b}(x) \approx y$$



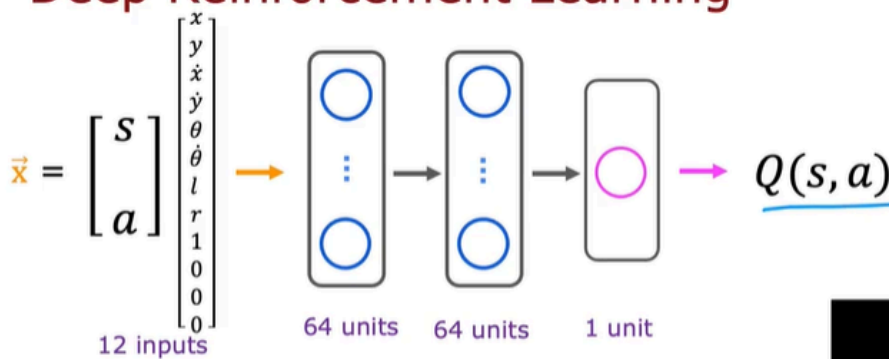
x, y

x'', y''

x^{10000}, y^{10000}

Algorithm refinement: Improved neural network architecture

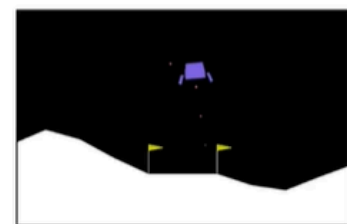
Deep Reinforcement Learning



In a state s , use neural network to compute

$Q(s, \text{nothing}), Q(s, \text{left}), Q(s, \text{main}), Q(s, \text{right})$

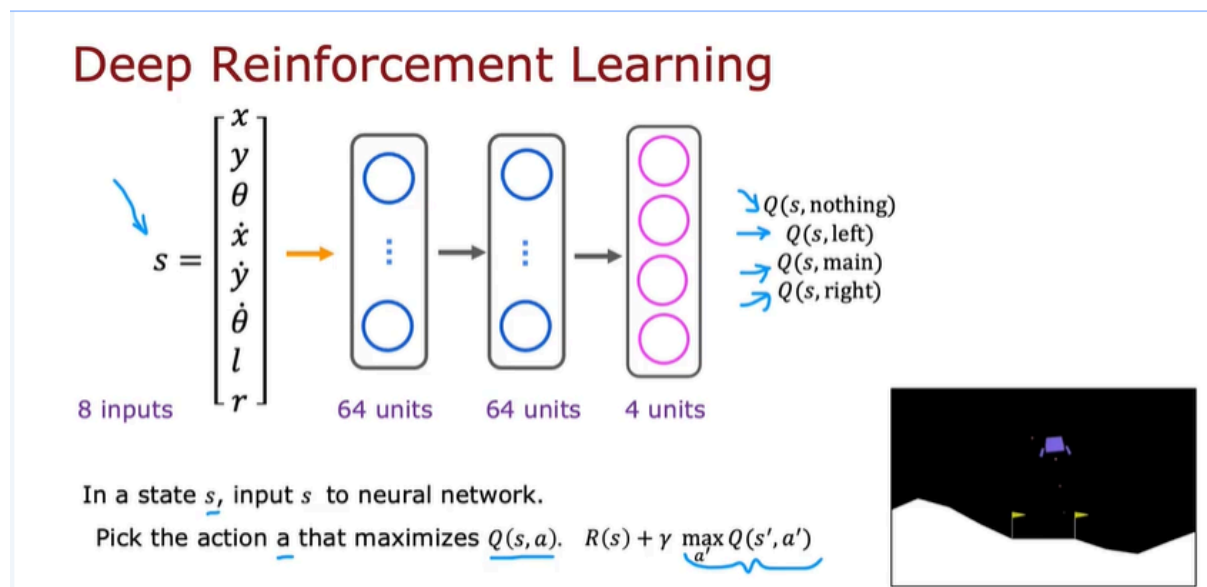
Pick the action a that maximizes $Q(s, a)$



This was the neural network architecture we saw previously, where it would input 12 numbers and output $Q(s, a)$. Whenever we are in some state s , we would have to carry out inference in the neural network separately four times to compute these four values so as to pick the action a that gives us the largest Q value.

This is inefficient because we have to carry our inference four times from every single state. Instead, it turns out to be more efficient to train a single neural

network to output all four of these values ($Q(s, \text{nothing})$, $Q(s, \text{left})$, ...) simultaneously.



This is what it looks like. Here's a modified neural network architecture where the input is eight numbers corresponding to the state of the lunar lander.

It then goes through the neural network with 64 units in the first hidden layer, 64 units in the second hidden layer. Now the output unit has four output units, and the job of the neural network is to have the four output units output $Q(s, \text{nothing})$, $Q(s, \text{left})$, $Q(s, \text{main})$, and $Q(s, \text{right})$.

The job of the neural network is to compute simultaneously the Q value for all four possible actions for when we are in the state s . This turns out to be more efficient because given the state s we can run inference just once and get all four of these values, and then very quickly pick the action a that maximizes $Q(s, a)$.

You notice also in Bellman's equations, there's a step in which we have to compute $\max$ over a prime Q of s prime a prime, this multiplied by γ and then there was plus R of s up here. This neural network also makes it much more efficient to compute this because we're getting Q of s prime a prime for all actions a prime at the same time. You can then just pick the $\max$ to compute this value for the right-hand side of Bellman's equations.

Algorithm refinement: ϵ (epsilon)-greedy policy

How to choose actions while still learning?

In some state s

Option 1:

Pick the action a that maximizes $Q(s, a)$.

Option 2:

- With probability 0.95, pick the action a that maximizes $Q(s, a)$. Greedy, "Exploitation"
- With probability 0.05, pick an action a randomly. "Exploration"

ϵ -greedy policy ($\epsilon = 0.05$)
0.95

$Q(s, \text{main})$ is low



Start ϵ high
 $1.0 \rightarrow 0.01$
Gradually decrease

The learning algorithm that we developed, even while you're still learning how to approximate $Q(s, a)$, you need to take some actions in the lunar lander. How do you pick those actions while you're still learning?

The most common way to do so is to use something called an Epsilon-greedy policy. Let's take a look at how that works. Here's the algorithm that you saw earlier. One of the steps in the algorithm is to take actions in the lunar lander.

When the learning algorithm is still running, we don't really know what's the best action to take in every state. If we did, we'd already be done learning. But even while we're still learning and don't have a very good estimate of $Q(s, a)$ yet, how do we take actions in this step of the learning algorithm? Let's look at some options. When you're in some state s , we might not want to take actions totally at random because that will often be a bad action. One natural option would be to pick whenever in state s , pick an action a that maximizes $Q(s, a)$. We may say, even if $Q(s, a)$ is not a great estimate of the Q function, let's just do our best and use our current guess of $Q(s, a)$ and pick the action a that maximizes it. It turns out this may work okay, but isn't the best option. Instead, here's what is commonly done.

Here's option 2, which is most of the time, let's say with probability of 0.95, pick the action that maximizes $Q(s, a)$. Most of the time we try to pick a good action using our current guess of $Q(s, a)$. But the small fraction of the time, let's say, 5% of the time, we'll pick an action a randomly. Why do we want to occasionally pick an action randomly? Well, here's why. Suppose there's some strange reason that $Q(s, a)$ was initialized randomly so that the learning algorithm thinks that firing the main thruster is never a good idea.

Maybe the neural network parameters were initialized so that $Q(s, \text{main})$ is always very low. If that's the case, then the neural network, because it's trying to pick the action a that maximizes $Q(s,a)$, it will never ever try firing the main thruster. Because it never ever tries firing the main thruster, it will never learn that firing the main thruster is actually sometimes a good idea.

Because of the random initialization, if the neural network somehow initially gets stuck in this mind that some things are bad idea, just by chance, then option 1, it means that it will never try out those actions and discover that maybe is actually a good idea to take that action, like fire the main thrusters sometimes. Under option 2 on every step, we have some small probability of trying out different actions so that the neural network can learn to overcome its own possible preconceptions about what might be a bad idea that turns out not to be the case.

This idea of picking actions randomly is sometimes called an exploration step. Because we're going to try out something that may not be the best idea, but we're going to just try out some action in some circumstances, explore and learn more about an action in the circumstance where we may not have had as much experience before. Taking an action that maximizes $Q(s,a)$, sometimes this is called a greedy action because we're trying to actually maximize our return by picking this.

Or in the reinforcement learning literature, sometimes you'll also hear this as an exploitation step. I know that exploitation is not a good thing, nobody should ever exploit anyone else. But historically, this was the term used in reinforcement learning to say, let's exploit everything we've learned to do the best we can.

In the reinforcement learning literature, sometimes you hear people talk about the exploration versus exploitation trade-off, which refers to how often do you take actions randomly or take actions that may not be the best in order to learn more, versus trying to maximize your return by say, taking the action that maximizes $Q(s,a)$.

This approach, that is option 2, has a name, is called an Epsilon-greedy policy, where here Epsilon is 0.05 is the probability of picking an action randomly. This is the most common way to make your reinforcement learning algorithm explore a little bit, even whilst occasionally or maybe most of the time taking greedy actions.

By the way, lot of people have commented that the name Epsilon-greedy policy is confusing because you're actually being greedy 95 percent of the time, not five percent of the time. So maybe $1 - \text{Epsilon}$ -greedy policy, because it's 95 percent greedy, five percent exploring, that's actually a more accurate description of the algorithm. But for historical reasons, the name Epsilon-greedy policy is what has stuck. This is the name that people use to refer to the policy that explores actually Epsilon fraction of the time rather than this greedy Epsilon fraction of the time.

Lastly, one of the trick that's sometimes used in reinforcement learning is to start off Epsilon high. Initially, you are taking random actions a lot at a time and then gradually decrease it, so that over time you are less likely to take actions randomly and more likely to use your improving estimates of the Q-function to pick good actions.

- For example, in the lunar lander exercise, you might start off with Epsilon very, very high, maybe even Epsilon equals 1.0. You're just picking actions completely at random initially and then gradually decrease it all the way down to say 0.01, so that eventually you're taking greedy actions 99 percent of the time and acting randomly only a very small one percent of the time. If this seems complicated, don't worry about it.

We'll provide the code in the Jupiter lab that shows you how to do this. If you were to implement the algorithm as we've described it with the more efficient neural network architecture and with an Epsilon-greedy exploration policy, you find that they work pretty well on the lunar lander. One of the things that I've noticed for reinforcement learning algorithm is that compared to supervised learning, they're more finicky in terms of the choice of hyper parameters.

- For example, in supervised learning, if you set the learning rate a little bit too small, then maybe the algorithm will take longer to learn. Maybe it takes three times as long to train, which is annoying, but maybe not that bad. Whereas in reinforcement learning, find that if you set the value of Epsilon not quite as well, or set other parameters not quite as well, it doesn't take three times as long to learn. It may take 10 times or a 100 times as long to learn. Reinforcement learning algorithms, I think because they're are less mature than supervised learning algorithms, are much more finicky to little choices of parameters like that, and it actually sometimes is frankly more frustrating to tune these parameters with reinforcement learning algorithm compared to a supervised learning algorithm.

But again, if you're worried about the practice lab, the program exercise, we'll give you a sense of good parameters to use in the program exercise so that you should be able to do that and successfully learn the lunar lander, hopefully without too many problems.

Algorithm refinement: Mini-batch and soft updates

- The first idea is called using mini-batches, and this turns out to be an idea they can both speedup your reinforcement learning algorithm and it's also applicable to supervised learning. They can help you speed up your supervised learning algorithm as well, like training a neural network, or training a linear regression, or logistic regression model.
- The second idea we'll look at is soft updates, which it turns out will help your reinforcement learning algorithm do a better job to converge to a good solution.

How to choose actions while still learning?

x	y
2104	400
1416	232
1534	315
852	178
...	...
3210	870

100,000,000

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^m (f_{w,b}(x^{(i)}) - y^{(i)})^2$$

$m = 100,000,000$

$m' = 1,000$

repeat {

$$w = w - \alpha \frac{\partial}{\partial w} \left(\frac{1}{2m'} \sum_{i=1}^{m'} (f_{w,b}(x^{(i)}) - y^{(i)})^2 \right)$$

$$b = b - \alpha \frac{\partial}{\partial b} \left(\frac{1}{2m'} \sum_{i=1}^{m'} (f_{w,b}(x^{(i)}) - y^{(i)})^2 \right)$$

}

There we had come up with this cost function for the parameters w and b , it was 1 over $2m$, sum of the prediction minus the actual value y^2 .

The gradient descent algorithm was to repeatedly update w as w minus the learning rate α times the partial derivative respect to w of the cost $J(w, b)$, and similarly to update b as follows. Let me just take this definition of $J(w, b)$ and substitute it in here. Now, when we looked at this example, way back when we were starting to talk about linear regression and supervised learning, the

training set size m was pretty small. I think we had 47 training examples. But what if you have a very large training set? Say m equals 100 million. There are many countries including the United States with over a 100 million housing units, and so a national census will give you a dataset that is this order of magnitude or size.

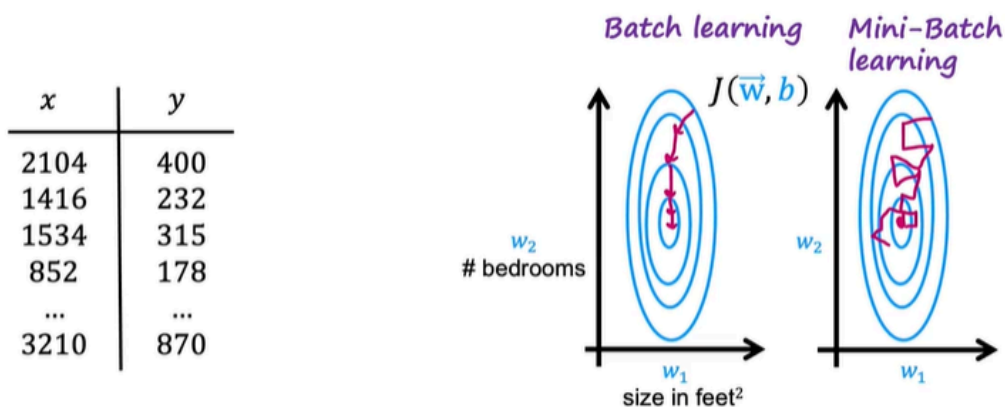
The problem with this algorithm when your dataset is this big, is that every single step of gradient descent requires computing this average over 100 million examples, and this turns out to be very slow. Every step of gradient descent means you would compute this sum or this average over 100 million examples. Then you take one tiny gradient descent step and you go back and have to scan over your entire 100 million example dataset again to compute the derivative on the next step, they take another tiny gradient descent step and so on and so on.

When the training set size is very large, this gradient descent algorithm turns out to be quite slow. The idea of mini-batch gradient descent is to not use all 100 million training examples on every single iteration through this loop. Instead, we may pick a smaller number, let me call it m prime equals say, 1,000. On every step, instead of using all 100 million examples, we would pick some subset of 1,000 or m prime examples. This inner term becomes $\frac{1}{2m \text{ prime}}$ is sum over sum m prime examples. Now each iteration through gradient descent requires looking only at the 1,000 rather than 100 million examples, and every step takes much less time and just leads to a more efficient algorithm. What mini-batch gradient descent does is on the first iteration through the algorithm, maybe it looks at that subset of the data. On the next iteration, maybe it looks at that subset of the data, and so on. For the third iteration and so on, so that every iteration is looking at just a subset of the data so each iteration runs much more quickly

Mini-batch



Mini-batch



You might remember with batch gradient descent, if these are the contours of the cost function J . Then batch gradient descent would say, start here and take a step, take a step, take a step, take a step, take a step. Every step of gradient descent causes the parameters to reliably get closer to the global minimum of the cost function here in the middle. In contrast, mini-batch gradient descent or a mini-batch learning algorithm will do something like this. If you start here, then the first iteration uses just five examples. It'll hit in the right direction but maybe not the best gradient descent direction.

Then the next iteration they may do that, the next iteration that, and that and sometimes just by chance, the five examples you chose may be an unlucky choice and even head in the wrong direction away from the global minimum, and so on and so forth. But on average, mini-batch gradient descent will tend

toward the global minimum, not reliably and somewhat noisily, but every iteration is much more computationally inexpensive and so mini-batch learning or mini-batch gradient descent turns out to be a much faster algorithm when you have a very large training set.

In fact, for supervised learning, where you have a very large training set, mini-batch learning or mini-batch gradient descent, or a mini-batch version with other optimization algorithms like Adam, is used more common than batch gradient descent.

Learning Algorithm

Initialize neural network randomly as guess of $Q(s, a)$.

Repeat {

Take actions in the lunar lander. Get $(s, a, R(s), s')$.

Store 10,000 most recent $(s, a, R(s), s')$ tuples.

← Replay Buffer

Train model:

1,000

Create training set of 10,000 examples using

$$x = (s, a) \text{ and } y = R(s) + \gamma \max_{a'} Q(s', a').$$

Train Q_{new} such that $Q_{new}(s, a) \approx y$.

Set $Q = Q_{new}$.



$x^{(1)}, y^{(1)}$
 $\vdots$
 $x^{(1000)}, y^{(1000)}$

Soft update

Soft Update

Set $Q = Q_{new}$. ← $Q(s, a)$
 $\nwarrow \nearrow$
 $W, B \quad W_{new}, B_{new}$

$$W = 0.01 W_{new} + 0.99 W$$

$$B = 0.01 B_{new} + 0.99 B$$

$$W = 1 W_{new} + 0 W$$

I've written out this step here of Set Q equals Q_{new} . But it turns out that this can make a very abrupt change to Q .

If you train a new neural network to new, maybe just by chance is not a very good neural network. Maybe is even a little bit worse than the old one, then you just overwrote your Q function with a potentially worse noisy neural network. The soft update method helps to prevent Q_{new} through just one unlucky step getting worse. In particular, the neural network Q will have some parameters, W and B , all the parameters for all the layers in the neural network.

This is called a soft update because whenever we train a new neural network W_{new} , we're only going to accept a little bit of the new value.

The state of reinforcement learning

Limitations of Reinforcement Learning

- Much easier to get to work in a simulation than a real robot!
- Far fewer applications than supervised and unsupervised learning.
- But ... exciting research direction with potential for future applications.

3. You have reached the final practice quiz of this class! What does that mean? (Please check all the answers, because all of them are correct!)

- ☒ You deserve to celebrate!
- ☒ Andrew sends his heartfelt congratulations to you!
- ☒ The DeepLearning.AI and Stanford Online teams would like to give you a round of applause!
- ☒ What an accomplishment -- you made it!